HH Stack Radar
Текст выступления

Радар навыков и авторизация. Промежуточные результаты разработки.
Ориентир: 8-10 минут, 16 слайдов.

Слайд 1. HH Stack Radar

Здравствуйте. Представляем промежуточные результаты разработки HH Stack Radar, сервиса анализа востребованности технологий по вакансиям. На первом блоке защиты мы описали проблему, архитектуру и план работ. На этом этапе показываем работающий интерфейс, сервис авторизации и радар навыков с фильтрацией по грейдам. Сразу обозначим границу результата: пользователи и авторизация работают через базу данных, а вакансии и требования пока генерируются для проверки аналитики. Это позволяет демонстрировать сценарии без зависимости от внешнего источника.

Слайд 2. Результаты относительно первого блока

В предыдущей презентации мы представили целевую архитектуру из фронтенда, сервиса авторизации, поиска и парсера. Сейчас реализованы фронтенд и auth-service. В интерфейсе доступны трекер публикаций и новый радар навыков. Авторизация поддерживает роли и права, а данные для графиков пока предоставляет временная заглушка. Поэтому мы не заявляем, что ежедневный сбор с hh.ru уже запущен или что накопили реальную историю рынка. Следующий шаг связан именно с подключением источника и выделением аналитики в search-service.

Слайд 3. Задача текущего этапа

Обычный график количества вакансий отвечает на вопрос, насколько востребовано направление. Радар навыков решает другую задачу: какие инструменты нужны внутри этого направления и как меняются требования. Для этого необходимо учитывать грейд. Например, Kubernetes может встречаться в требованиях к Senior, но отсутствовать у Middle в нашем демонстрационном сценарии. Мы добавили грейд и стек в модель каждой вакансии, согласовали фильтры между аналитикой и списком публикаций и подготовили историю для проверки изменений во времени.

Слайд 4. Реализованные возможности

В текущем прототипе есть два раздела аналитики. Трекер вакансий показывает количество новых публикаций по Java и React. Радар навыков позволяет выбрать направление, грейд и период, а затем посмотреть динамику долей, сравнение двух завершённых периодов и весь стек в таблице. Для графиков можно выбрать до шести навыков. В списке последних вакансий видны грейд и требования. Оба режима запуска используют один контракт: полностью автономные моки во фронтенде или Java-заглушку вместе с настоящей авторизацией.

Слайд 5. Текущая архитектура

Браузер обращается к Next.js через один origin. В основном режиме Next.js проксирует API-запросы к сервису на Spring Boot. Auth-service проверяет вход, выпускает access-токен и управляет refresh-сессиями. Пользователи, роли и сессии хранятся в PostgreSQL, схему создаёт Flyway. Аналитические маршруты пока обслуживает временный пакет mock внутри Java-сервиса. Когда появится search-service, эти маршруты перейдут в него. Parser-service должен будет получать данные hh.ru. На схеме будущие компоненты выделены отдельно, чтобы не смешивать план и текущую реализацию.

Слайд 6. Авторизация и права доступа

В сервисе реализована накопительная ролевая модель: аналитик получает права пользователя, администратор получает права аналитика. Контроллеры проверяют конкретные права, например STATS_VIEW для аналитики. Пароли хранятся в виде BCrypt-хешей. Access-токен подписывается RSA и живёт пятнадцать минут, фронтенд хранит его в памяти вкладки. Refresh-токен находится в httpOnly cookie и ротируется при обновлении сессии. Для аналитического API проверены ответы 401 без входа и 403 без нужного права. Часть прав уже заведена под будущие функции, но наличие права ещё не означает готовность соответствующего экрана.

Слайд 7. Модель вакансии и требований

Раньше грейд использовался внутри генератора преимущественно для выбора зарплаты. Теперь grade входит в ответ API и участвует в фильтрации. Поле skills содержит идентификаторы навыков, а каталог задаёт их человекочитаемые названия. Это отделяет данные от подписей в интерфейсе и подготавливает основу для нормализации синонимов. Для исторической аналитики используется publishedAt. Мы считаем требования на дату публикации. Отдельную историю редактирования одного и того же объявления текущая модель пока не ведёт, поэтому не подменяем эту задачу графиком новых публикаций.

Слайд 8. Моковые данные по грейдам

Моковый набор содержит публикации за три года и два направления. Генератор связывает название вакансии с грейдом, диапазоном зарплаты и набором требований. В демонстрационных правилах Kubernetes и проектирование систем встречаются только у Senior и Lead. У Junior ограничен набор инфраструктурных инструментов. Вероятности отдельных навыков меняются по времени: например, TypeScript становится чаще, Redux реже. Это заданные сценарии для тестирования, а не выводы о рынке. Генерация навыков согласована между TypeScript и Java, включая контрольную последовательность в тестах.

Слайд 9. Методика расчёта

Основная метрика — доля вакансий, в которых требуется выбранный навык. В числителе находится число таких вакансий, в знаменателе — вся выборка за тот же период, направление и грейд. Например, семьдесят вакансий из ста дают семьдесят процентов. Если раньше было шестьдесят процентов, изменение составляет плюс десять процентных пунктов. Вакансия учитывается один раз на навык, даже если он продублирован. Пустая выборка возвращает null, а отсутствие навыка в непустой выборке — ноль. Текущий незавершённый период отображается в истории, но исключается из сравнения.

Слайд 10. Радар требований по грейду

На слайде показан пример из текущего мокового набора. Радар сравнивает доли навыков за два завершённых месяца для Java Senior. Для Kubernetes в октябре получилась доля около семидесяти четырёх процентов: девяносто одна вакансия из ста двадцати трёх. В сентябре было примерно шестьдесят восемь процентов. При переключении на Middle доля Kubernetes становится нулевой по заданному правилу генерации. Значения описывают синтетический набор. Такой пример подтверждает, что фильтр грейда меняет и числитель, и знаменатель и не смешивает требования разных уровней.

Слайд 11. Динамика требований во времени

Линейный график дополняет радар. Он показывает последовательность периодов, а не только сравнение двух точек. Для React Senior мы вывели TypeScript, Next.js и Redux. Все линии используют одинаковый знаменатель внутри каждой временной корзины. Видимые колебания зависят и от вероятностей генератора, и от размера месячной выборки. Поэтому одно изменение между соседними месяцами нельзя автоматически называть устойчивым трендом. На реальных данных нужно будет учитывать объём выборки и качество распознавания навыков. Сейчас график демонстрирует корректность хранения, группировки и отображения истории.

Слайд 12. API и режимы интеграции

Для радара добавлен маршрут vacancies/skills. Он принимает период, направления и грейды и возвращает каталог навыков, временные корзины, общий размер выборки и сводку. Список вакансий принимает те же направления и грейды. Контракт описан в OpenAPI и обновлён в обоих проектах. Неизвестные или пустые значения фильтров возвращают 400. В режиме USE_MOCKS=true обработчики Next.js работают автономно. В основном режиме запросы идут через proxy к Java. Так мы можем отдельно проверять интерфейс и интеграцию с реальной авторизацией, сохраняя формат ответов.

Слайд 13. Сценарий демонстрации

Демонстрацию удобно начать со входа и существующего трекера вакансий. Затем открываем радар, выбираем Java и Middle и показываем нулевую долю Kubernetes. После переключения на Senior меняются таблица требований, графики и список вакансий. Для Kubernetes можно сравнить завершённые месяцы, а затем переключить гранулярность на неделю или год. Завершаем просмотром конкретной вакансии с грейдом и стеком. Сначала показываем границы мокового набора, чтобы комиссия оценивала работу механизма, а не воспринимала демонстрационные числа как реальную рыночную статистику.

Слайд 14. Проверки реализации

Для фронтенда подготовлено двенадцать тестов: они проверяют агрегацию, доли, фильтры и ответы API. Для Java есть семь сценариев, одинаковые в двух копиях сервиса. В них проверяются завершённые периоды, пустые выборки, удаление дублей навыков, грейды и права доступа. Тесты помогли обнаружить две ошибки: отказ в доступе обрабатывался как 500, а последовательные идентификаторы создавали искусственные скачки требований в генераторе. Обе ошибки исправлены. Также выполнены проверка типов, production-сборка и просмотр интерфейса. Эти проверки ещё не подтверждают качество будущего парсера или готовность к промышленной нагрузке.

Слайд 15. Следующий этап разработки

Главный следующий шаг — заменить синтетические данные реальными вакансиями. Для этого нужен parser-service с сохранением публикаций и обработкой повторного получения данных. Затем необходимо нормализовать названия навыков и проверить качество их извлечения на ручной разметке. Аналитику следует выделить в search-service, сохранив уже согласованный контракт. После этого можно расширять направления и добавлять сравнение компаний с рынком. Также остаются интерфейсы администрирования, работа с экспортом и эксплуатационные проверки. На этом этапе отдельно нужно актуализировать зависимости и довести сквозные проверки с PostgreSQL.

Слайд 16. Итог текущего этапа

По итогам этапа у вакансии появился явный грейд и стек, а в интерфейсе — отдельный радар навыков. Система считает доли требований, показывает их изменение во времени и согласованно фильтрует последние вакансии. Реализованы авторизация и проверка прав, контракт поддерживают фронтенд и Java-заглушка. При этом данные аналитики остаются синтетическими, а реальный сбор и нормализация входят в следующий этап. Полученный результат позволяет проверить пользовательские сценарии и правила расчёта до подключения источника. Спасибо за внимание, готовы ответить на вопросы.

Ответы на возможные вопросы

Почему пока используются моки?

Они позволяют независимо проверить интерфейс, формулы, грейды и временные периоды. Реальный сбор ещё не подключён. После его появления контракт API можно сохранить, заменив источник данных.

Откуда берётся грейд?

Сейчас его явно задаёт генератор вместе с названием вакансии. Автоматическое определение грейда из реальных объявлений ещё предстоит реализовать и оценить на ручной разметке.

Почему у Middle нет Kubernetes?

Это условие демонстрационного набора для проверки фильтра. В реальном рынке такая комбинация возможна. Будущий парсер должен сохранять обнаруженные требования, а не навязывать правило из моков.

Чем процентные пункты отличаются от процентов?

Изменение с 60% до 70% составляет +10 процентных пунктов. Относительный рост был бы около 16,7%. В таблице требований показаны именно процентные пункты.

Что означает нулевая доля?

В непустой выборке ни одна вакансия не требует навык. Если сама выборка пустая, API возвращает null, чтобы не представлять отсутствие наблюдений как нулевой спрос.

Хранится ли история изменений одной вакансии?

Пока история строится по требованиям на дату публикации. Версии редактирования одного объявления и их сравнение не реализованы.

Зачем аналитика находится внутри auth-service?

Это временный пакет mock для работы фронтенда с настоящей авторизацией. Целевая архитектура предусматривает перенос аналитических маршрутов в search-service.

Какие проверки доступа реализованы?

Защищённый API требует аутентификацию и конкретное право, для аналитики STATS_VIEW. Проверены 401 без входа и 403 при отсутствии права. Пользовательские экраны для всех административных функций ещё предстоит завершить.

Можно ли по показанным графикам делать выводы о рынке?

Нет, показанные числа получены из генератора. Они демонстрируют работу расчётов. Рыночные выводы требуют реальных данных, достаточной выборки и проверки качества извлечения навыков.